Untitled79

March 17, 2026

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.manifold import TSNE,trustworthiness
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score,davies_bouldin_score
```

[2]:
```
data = pd.read_csv(r"C:/Users/USER/Documents/data.csv")
data.head()
```

[2]:
```
     Samples  gene_0    gene_1    gene_2    gene_3     gene_4  gene_5  \
0  sample_0     0.0  2.017209  3.265527  5.478487  10.431999       0
1  sample_1     0.0  0.592732  1.588421  7.586157   9.623011       0
2  sample_2     0.0  3.511759  4.327199  6.881787   9.870730       0
3  sample_3     0.0  3.663618  4.507649  6.659068  10.196184       0
4  sample_4     0.0  2.655741  2.821547  6.539454   9.738265       0

     gene_6    gene_7  gene_8  …  gene_16373  gene_16374  gene_16375  \
0  7.175175  0.591871     0.0  …    8.750533    7.421257    4.692126
1  6.816049  0.000000     0.0  …    6.638879    7.991732    5.709045
2  6.972130  0.452595     0.0  …    8.205754   10.375778    1.839758
3  7.843375  0.434882     0.0  …    8.093185    8.424771    5.502251
4  6.566967  0.360982     0.0  …    7.522228   12.176650   10.305423

   gene_16376  gene_16377  gene_16378  gene_16379  gene_16380  gene_16381  \
0    1.334282    1.650856    3.973382    0.000000    9.291933    0.840926
1    0.811142    2.717824    3.255773    1.590818    3.467410    1.178045
2    0.000000    3.004897    0.796598    0.000000    5.533710    0.225892
3    0.434882    2.207862    1.039419    0.000000    8.358278    0.377401
4    0.360982    1.099497    0.360982    0.649386    9.315607    1.052833

   gene_16382
0    7.886642
1    8.864223
```

```
2    9.843799
3    7.701320
4   10.646325

[5 rows x 16384 columns]
```

[3]:
```python
data.shape
```

[3]:
```
(801, 16384)
```

[4]:
```python
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 801 entries, 0 to 800
Columns: 16384 entries, Samples to gene_16382
dtypes: float64(16351), int64(32), object(1)
memory usage: 100.1+ MB
```

[5]:
```python
data.describe()
```

[5]:
```
           gene_0      gene_1      gene_2      gene_3      gene_4  gene_5  \
count  801.000000  801.000000  801.000000  801.000000  801.000000   801.0
mean     0.026642    3.010909    3.095350    6.722305    9.813612     0.0
std      0.136850    1.200828    1.065601    0.638819    0.506537     0.0
min      0.000000    0.000000    0.000000    5.009284    8.435999     0.0
25%      0.000000    2.299039    2.390365    6.303346    9.464466     0.0
50%      0.000000    3.143687    3.127006    6.655893    9.791599     0.0
75%      0.000000    3.883484    3.802534    7.038447   10.142324     0.0
max      1.482332    6.237034    6.063484   10.129528   11.355621     0.0

           gene_6      gene_7      gene_8      gene_9  …  gene_16373  \
count  801.000000  801.000000  801.000000  801.000000  …  801.000000
mean     7.405509    0.499882    0.016744    0.013428  …    7.603795
std      1.108237    0.508799    0.133635    0.204722  …    0.946110
min      3.930747    0.000000    0.000000    0.000000  …    4.343664
25%      6.676042    0.000000    0.000000    0.000000  …    7.138722
50%      7.450114    0.443076    0.000000    0.000000  …    7.627585
75%      8.121984    0.789354    0.000000    0.000000  …    8.143271
max     10.718190    2.779008    1.785592    4.067604  …   12.009825

       gene_16374  gene_16375  gene_16376  gene_16377  gene_16378  gene_16379  \
count  801.000000  801.000000  801.000000  801.000000  801.000000  801.000000
mean     9.353000    6.590923    0.541963    3.142053    1.710593    2.912340
std      1.098303    2.126161    0.826818    3.210868    1.473575    3.673102
min      6.321394    1.839758    0.000000    0.000000    0.000000    0.000000
25%      8.668126    4.990873    0.000000    1.209079    0.631058    0.410233
50%      9.211976    6.036009    0.360982    1.993928    1.371280    1.169155
75%      9.970681    8.346159    0.816395    3.078627    2.413865    3.996832
```

```
max     14.576024   11.651756    9.770221   11.940079    8.041873   13.842124

       gene_16380  gene_16381  gene_16382
count  801.000000  801.000000  801.000000
mean     7.229904    1.250715    9.659116
std      3.239120    1.568535    1.073619
min      0.000000    0.000000    6.847371
25%      4.733490    0.000000    8.894987
50%      7.622132    0.710790    9.555136
75%     10.010011    1.957655   10.303804
max     13.875874    8.039901   13.515601

[8 rows x 16383 columns]
```

[6]:
```
data.isnull().sum()
```

[6]:
```
Samples       0
gene_0        0
gene_1        0
gene_2        0
gene_3        0
              ..
gene_16378    0
gene_16379    0
gene_16380    0
gene_16381    0
gene_16382    0
Length: 16384, dtype: int64
```

[7]:
```
data_encoded = data.drop(columns="Samples")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(data_encoded)
```

[8]:
```
pca_full = PCA(n_components=None)
pca_full.fit(X_scaled)
```

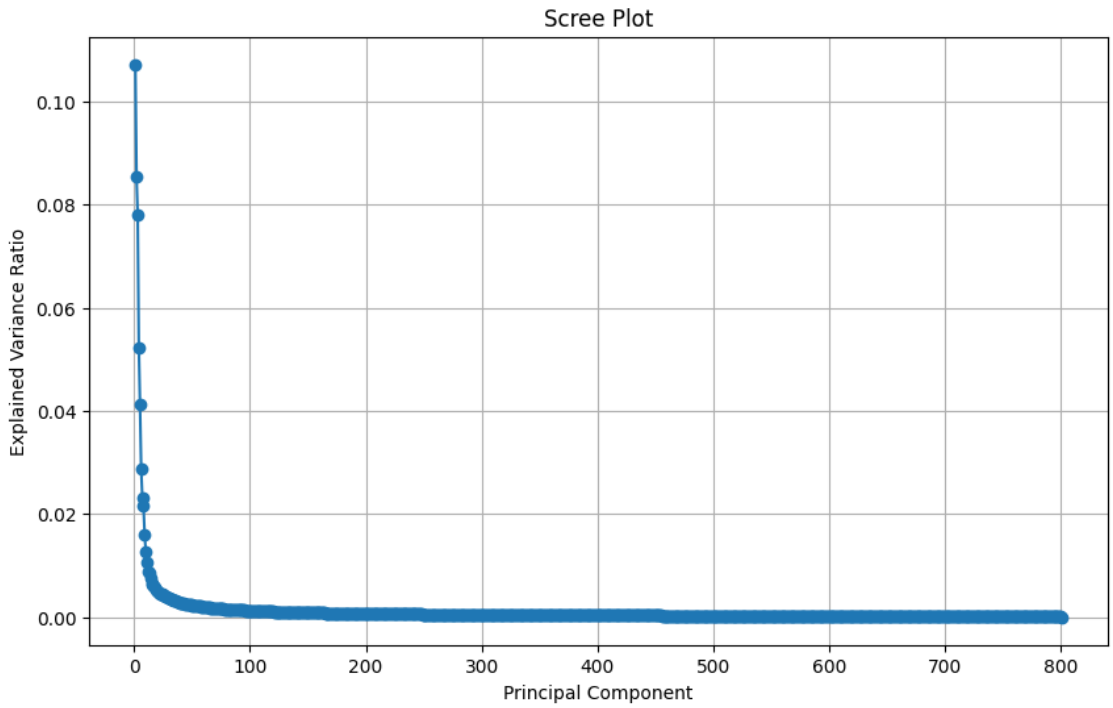
[8]:
```
PCA()
```

[9]:
```
explained_variance = pca_full.explained_variance_ratio_
print(explained_variance[:10])
```

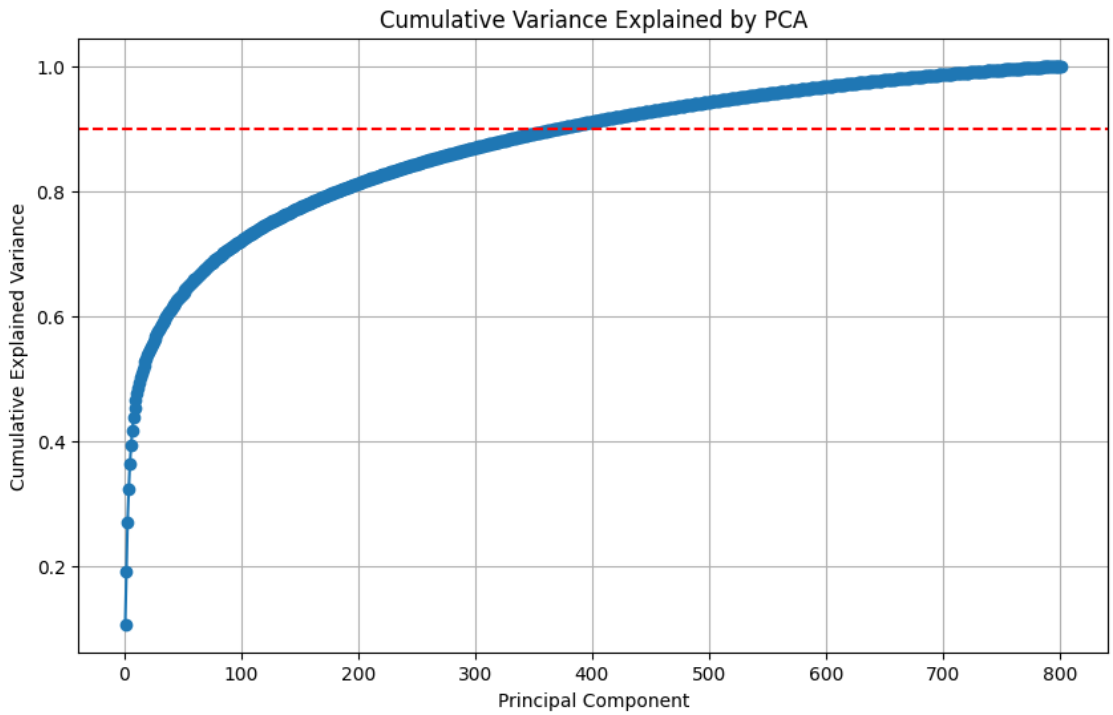
```
[0.10715262 0.08534221 0.0779273  0.0521833  0.04129962 0.02881527
 0.02323829 0.0215801  0.01612736 0.0126549 ]
```

[10]:
```
plt.figure(figsize=(10,6))
plt.plot(np.arange(1, len(explained_variance)+1), explained_variance,␣
 ↪marker='o')
```

```
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot")
plt.grid(True)
plt.show()
```



[11]:
```
cumulative_variance = np.cumsum(explained_variance)
plt.figure(figsize=(10,6))
plt.plot(np.arange(1, len(cumulative_variance)+1), cumulative_variance,␣
 ↪marker='o')
plt.xlabel("Principal Component")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Variance Explained by PCA")
plt.grid(True)
plt.axhline(y=0.9, color='r', linestyle='--')  # 90% variance line
plt.show()
```



[13]:
```
pca1 = PCA(n_components=350)
data_pca = pca.fit_transform(X_scaled)
```

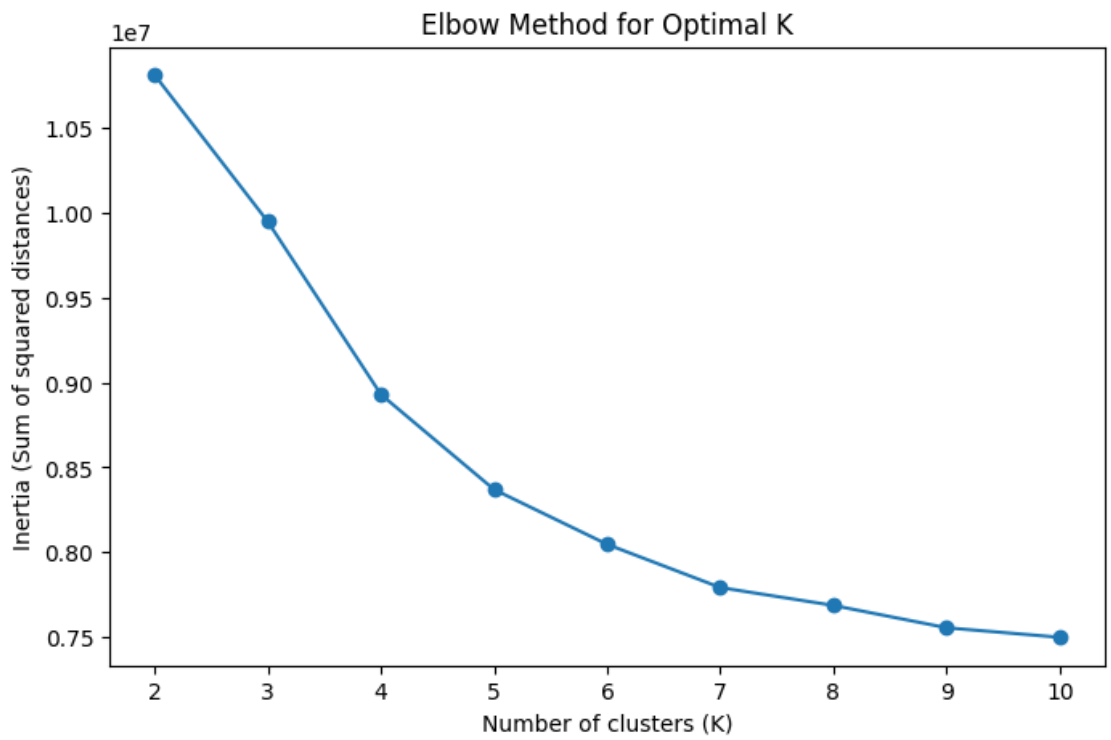
[14]:
```
data_pca.shape
```

[14]: (801, 350)

[15]:
```
inertia_list = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data_pca)
    inertia_list.append(kmeans.inertia_)
```

[16]:
```
plt.figure(figsize=(8,5))
plt.plot(K_range, inertia_list, marker='o')
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia (Sum of squared distances)")
plt.title("Elbow Method for Optimal K")
plt.show()
```



[17]:
```
kmeans = KMeans(n_clusters=5, random_state=42)
cluster_labels = kmeans.fit_predict(data_pca)
```

[18]:
```
pd.Series(cluster_labels).value_counts()
```

[18]:
```
4    251
0    198
3    145
2    134
1     73
Name: count, dtype: int64
```

[19]:
```
sil_score = silhouette_score(data_pca, cluster_labels)
print("Silhouette Score:", sil_score)
```

```
Silhouette Score: 0.16158903194230426
```

[20]:
```
db_score = davies_bouldin_score(data_pca, cluster_labels)
print("Davies-Bouldin Index:", db_score)
```

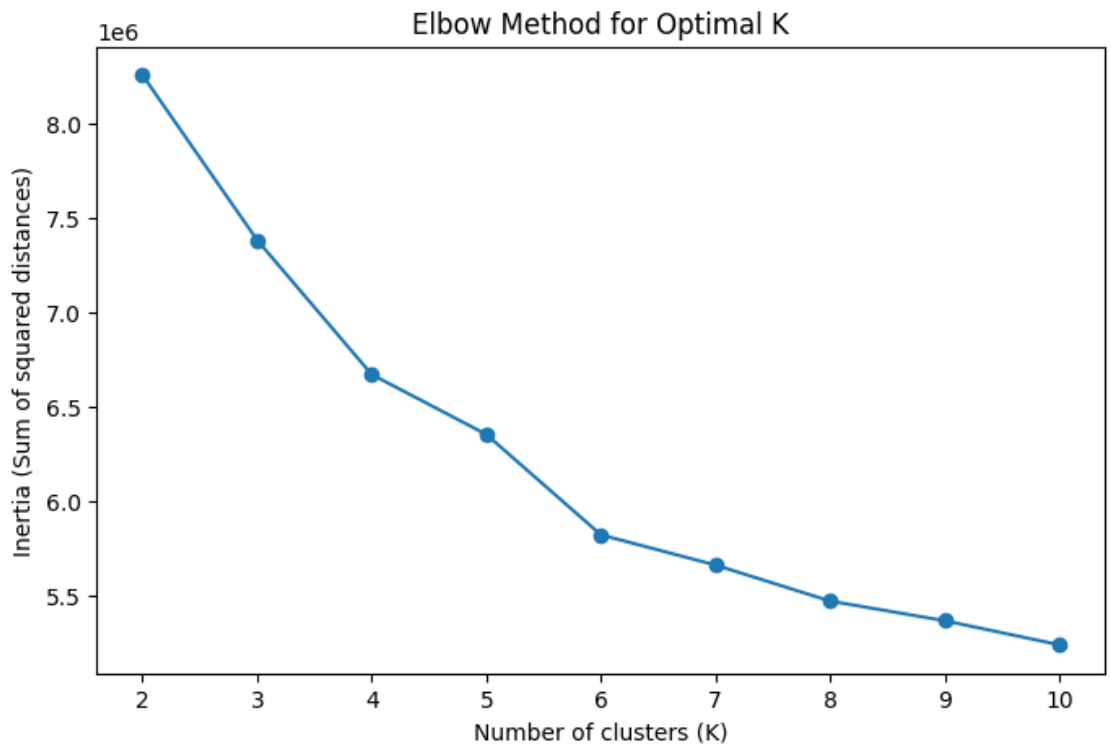
```
Davies-Bouldin Index: 2.430192840753834
```

[59]:
```
pca2 = PCA(n_components=100)
data_pca2 = pca.fit_transform(X_scaled)
```

[60]:
```
inertia_list = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data_pca2)
    inertia_list.append(kmeans.inertia_)
```

[61]:
```
plt.figure(figsize=(8,5))
plt.plot(K_range, inertia_list, marker='o')
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia (Sum of squared distances)")
plt.title("Elbow Method for Optimal K")
plt.show()
```



[62]:
```
kmeans = KMeans(n_clusters= 6, random_state=42)
cluster_labels = kmeans.fit_predict(data_pca2)
```

[63]:
```
sil_score = silhouette_score(data_pca2, cluster_labels)
print("Silhouette Score:", sil_score)
```

```
Silhouette Score: 0.21146175533428224
```

[64]:
```
db_score = davies_bouldin_score(data_pca2, cluster_labels)
print("Davies-Bouldin Index:", db_score)
```

```
Davies-Bouldin Index: 1.9459128854596452
```

[65]:
```
tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate=200,
    max_iter=1000,
    random_state=42
)

tsne_result = tsne.fit_transform(data_pca)
```

[66]:
```
tsne_result.shape
```

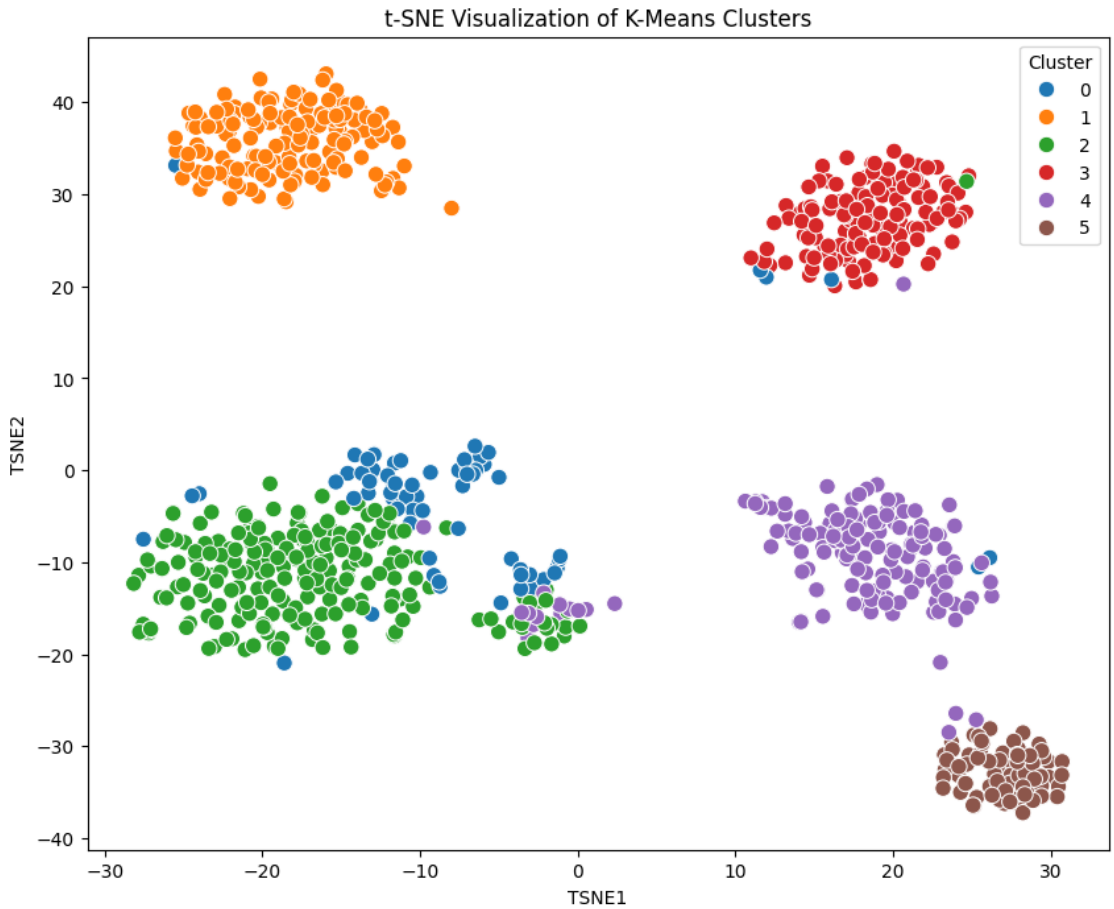
[66]:
```
(801, 2)
```

[67]:
```
tsne_df = pd.DataFrame()
tsne_df["TSNE1"] = tsne_result[:,0]
tsne_df["TSNE2"] = tsne_result[:,1]
tsne_df["Cluster"] = cluster_labels
```

[68]:
```
plt.figure(figsize=(10,8))

sns.scatterplot(
    x="TSNE1",
    y="TSNE2",
    hue="Cluster",
    palette="tab10",
    data=tsne_df,
    s=80
)

plt.title("t-SNE Visualization of K-Means Clusters")
plt.show()
```



[69]:
```
tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate=200,
    max_iter=1000,
    random_state=42
)

tsne_result = tsne.fit_transform(data_pca2)
```

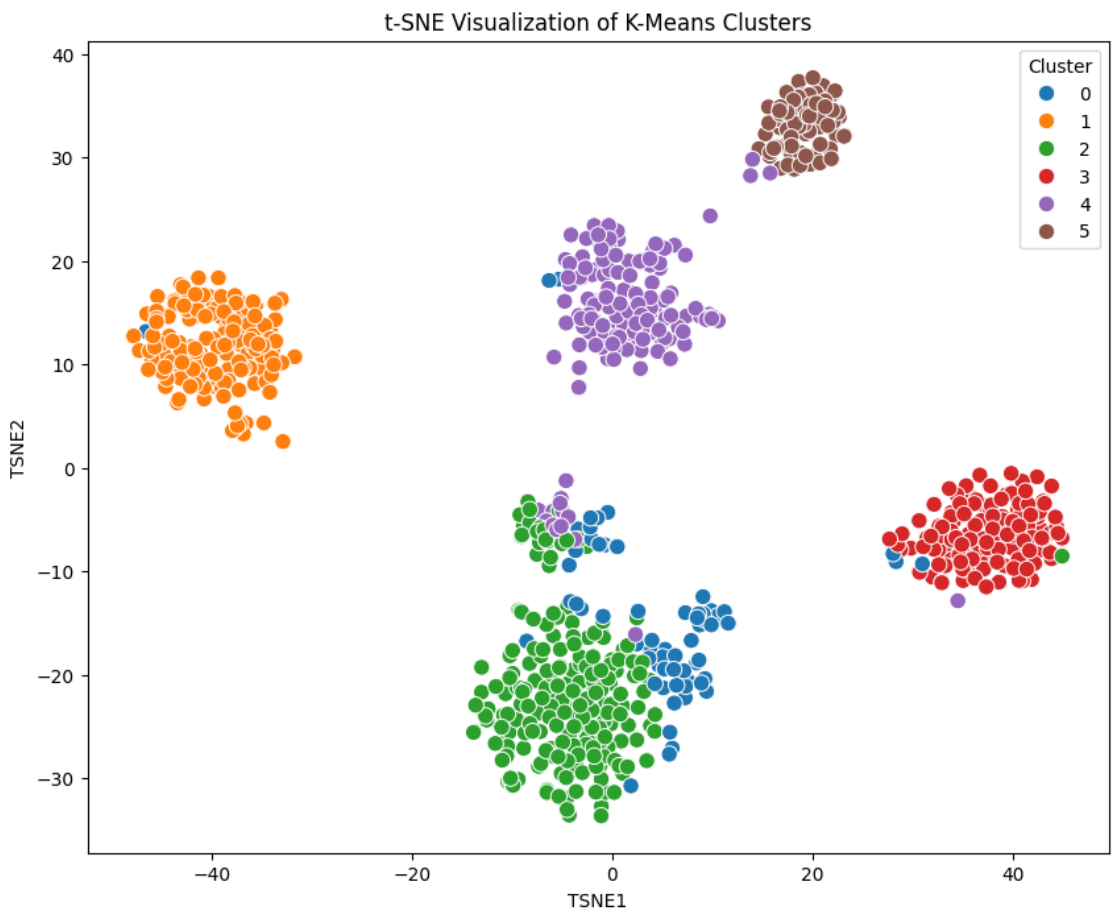
[70]:
```
tsne_df = pd.DataFrame()
tsne_df["TSNE1"] = tsne_result[:,0]
tsne_df["TSNE2"] = tsne_result[:,1]
tsne_df["Cluster"] = cluster_labels
```

[71]:
```
plt.figure(figsize=(10,8))

sns.scatterplot(
```

```
    x="TSNE1",
    y="TSNE2",
    hue="Cluster",
    palette="tab10",
    data=tsne_df,
    s=80
)

plt.title("t-SNE Visualization of K-Means Clusters")
plt.show()
```



[ ]:

```
```